Architecture Teardown: The Simog Ecosystem

Module-by-Module Structural Analysis and Dependency Mapping

AUTORITÀ
NAZIONALE
ANTICORRUZIONE

The System at a Glance

Centralized Core

SimogCommon acts as a monolithic dependency hub, centralizing JDBC access and business logic.

Machine-to-Machine Heavy

The system functions as a massive API engine, exposing broad SOAP interfaces for external integration.

Architectural Sediment

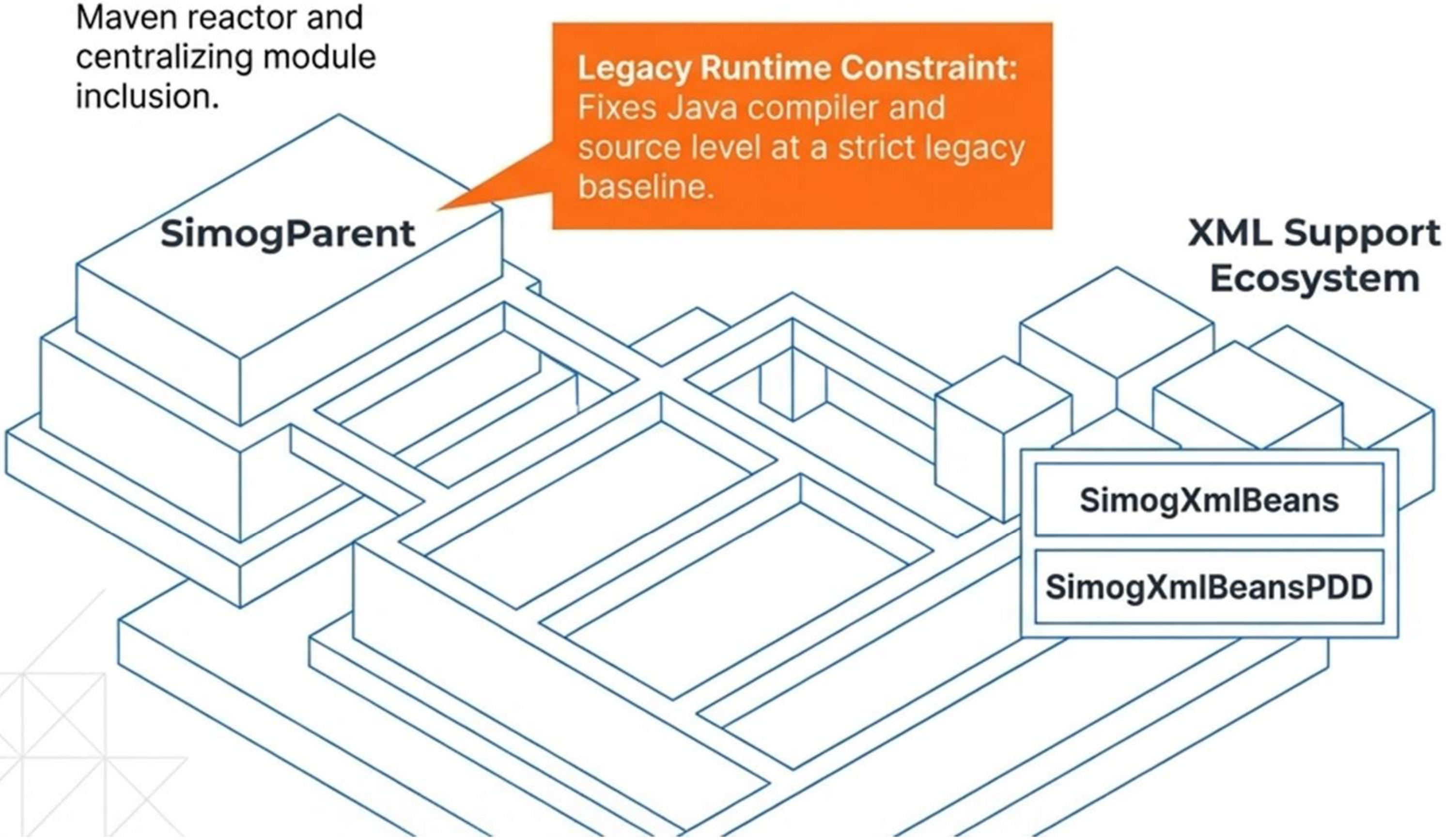
The codebase houses co-existing legacy and modern patterns, notably transitioning from custom JDBC/SQL to JPA in bounded subsystems.

The Simog Architectural Topology

End-User & API Surfaces	SimogWeb, SimogMassLoader, SimogWSPDD, SimogWSAGG, SimogWSTED
Orchestration & Control	SimogWSCommon, SimogAVCPLogin, SimogFlusso
The Core Domain	SimogCommon, SimogTEDCommon
Foundation & Build	SimogParent, SimogXmlBeans, SimogXmlBeansPDD

Foundation & Build Constraints

Root POM aggregating the Maven reactor and centralizing module inclusion.

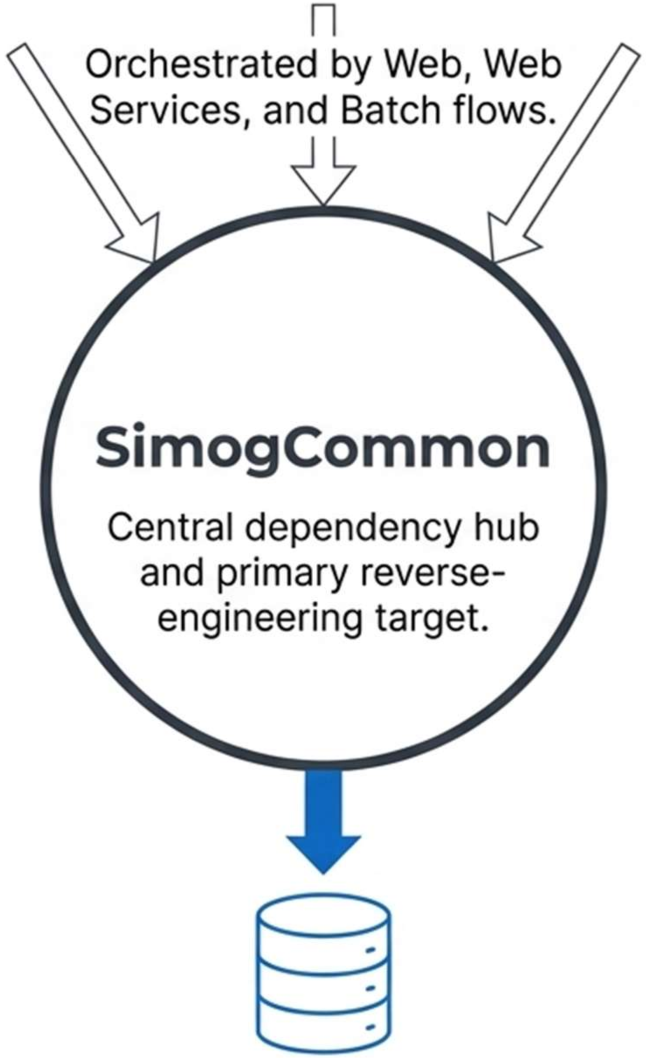


Schema-derived support classes used for XML serialization and validation. Built via Ant/schema tools and heavily utilized by the SOAP/XML handling layers.

The Monolithic Core: SimogCommon

Internal Node Responsibilities

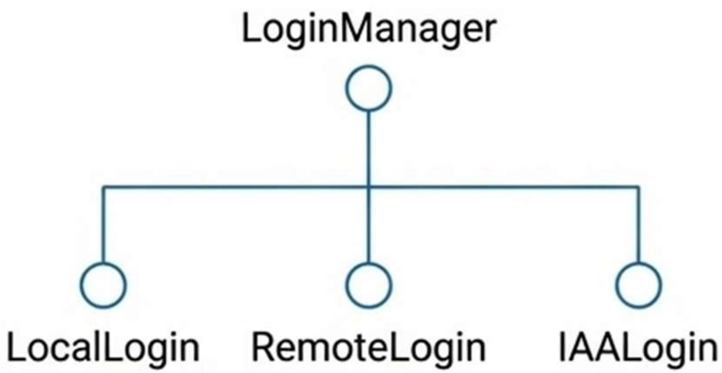
- Central business rules & validation (WorkFlowController)
- Custom DB access & SQL generation (AccessiDB)
- Utility & configuration (SimogProperties)



Houses managers for Tenders (GaraManager), Lots (LottoManager), Publications, Documents, and Administrative Structures.

Authentication & Flow Control

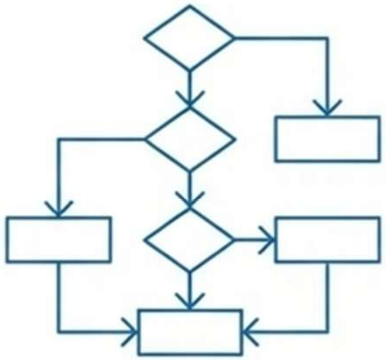
SimogAVCPLogin



Abstracts login mode selection and connects to remote IAM services.

Insight: Coexistence of old and new login proxy paths (Local vs. IAA) reveals visible architectural sediment from authentication evolution.

SimogFlusso

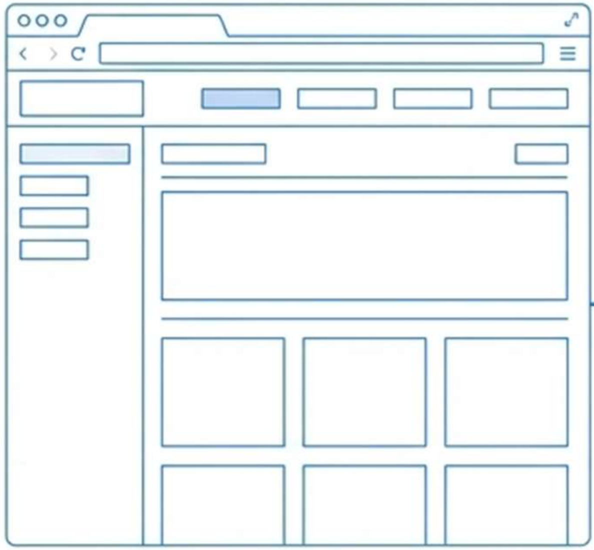


Control flow selection and workflow abstraction for operational scenarios.

Key Class: WorkFlowController.

Insight: Small footprint, but establishes workflow orchestration as a first-class architectural concern.

The End-User Surface: SimogWeb



1 **Browser HTTP Request:** End-user navigation and data-entry UI.

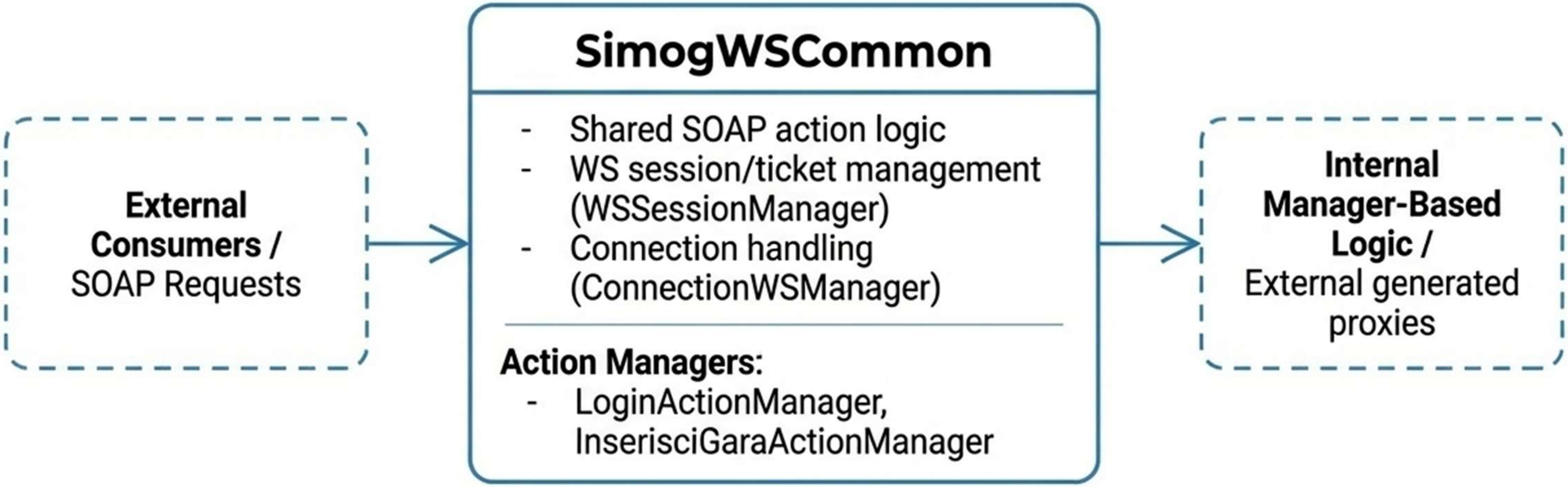
2 **Servlet Dispatch:** Flows through functional servlets mapped in web.xml (e.g., ServletBase, SrvAutentica, SrvInserisciGara).

3 **JSP Rendering & Session Management:** Handles UI presentation and session-managed business operations.

4 **Core Handoff:** Dispatches to SimogCommon and Database.

Observation: Broad legacy web-app style encompassing both low-level HTTP infrastructure and direct business-facing controller logic.

The Interoperability Hub: SimogWSCommon



Insight: Bridges rigid external SOAP contract semantics with internal domain orchestrators, centralizing all technical session handling for service consumers.

Service Exposure Matrix

Module	Scope	Integration Target	Complexity	Key Interface
SimogWSPDD	Broad & Operation-Heavy	High-contract machine-to-machine procurement	High	SimogWSPDD
SimogWSAGG	Specialized	Aggregator & Initiative flows	Medium	SimogWSAGG
SimogWSTED	Bounded Context	TED publication & delta management	Medium	SimogWSTED

Observation: This architecture reveals a system designed as much as an API engine as a user-facing web application.

Operational Batch Subsystem: SimogMassLoader

1. CLI Invocation (it.avlp.simog.massload.Main)
2. Schema Validation (Leveraging SimogXmlBeans)
3. Directory Processing (File-system ingestion)
4. DB Ingestion (Handoff to SimogCommon)
5. Feedback Output (Result production)

Observation Note: Operationally significant batch integration—not merely a hidden internal utility, but a primary mechanism for procurement XML flows.

The TED Subsystem: A Bounded Context

SimogTEDCommon	SimogWSTED
Role: TED-specific entities, services, validators, and XML support. **Key Mechanics:** Explicit persistence model using JPA (persistence.xml, TEDDbService).	**Role:** Exposes SOAP operations for verification, publication, and cancellation. **Integration:** Strictly manages TED delta flows.

Key Insight: Forms a relatively clear, separate integration boundary with highly structured internal logic compared to the legacy core.

Architectural Evolution: Legacy vs. Modern

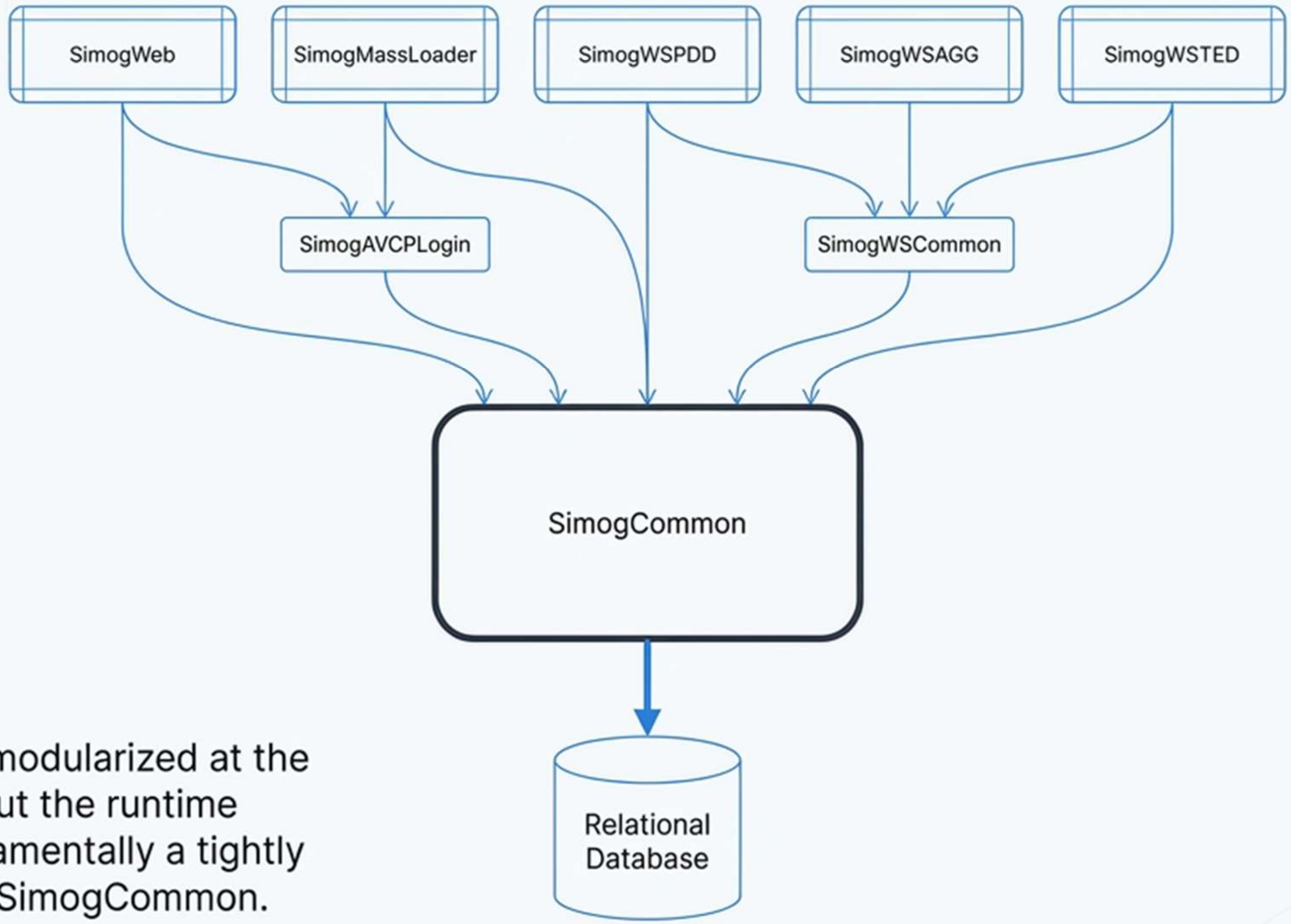
The Legacy Core: SimogCommon

1. Architecture: Highly coupled, centralized manager pattern.
2. Persistence: Raw **JDBC** access with custom SQL string generation (AccessiDB).
3. Data Model: Implicit data structures managed procedurally.

The Modern Shift: TEDCommon

1. Architecture: Bounded context, segregated service logic.
2. Persistence: **JPA** entity mapping and structured repositories (TEDDbService).
3. Data Model: Explicit persistence model governed by persistence.xml.

The Macro Dependency Flow Model



Takeaway: Code is modularized at the Maven build level, but the runtime architecture is fundamentally a tightly coupled funnel into SimogCommon.

Key Architectural Observations

1. Visible Sedimentation

The codebase acts as an archaeological dig. Authentication paths (coexisting local, remote, and IAA proxies in SimogAVCPLogin) reveal partially completed architectural transitions.

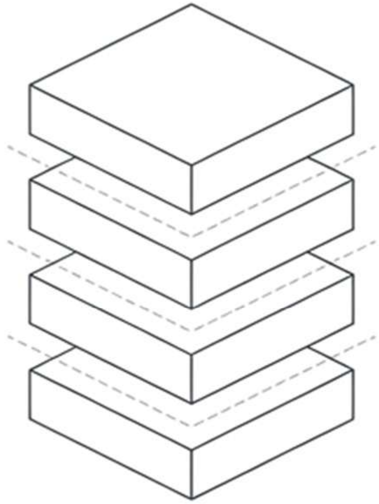
2. The API Engine Reality

Despite the presence of SimogWeb, the massive footprint of WSPDD, WSAGG, and WSCommon proves this system is functionally an integration-heavy M2M engine.

3. The Illusion of Modularity

While SimogParent cleanly segregates 13 build modules, the runtime execution is tightly coupled. SimogCommon remains a monolithic dependency hub that must be unraveled carefully.

The Simog Baseline Established



Simog is a robust, integration-heavy procurement engine. Modernization requires respecting its vast machine-to-machine SOAP contracts while progressively migrating the SimogCommon JDBC core toward the bounded, JPA-driven patterns modeled in the TED subsystem.